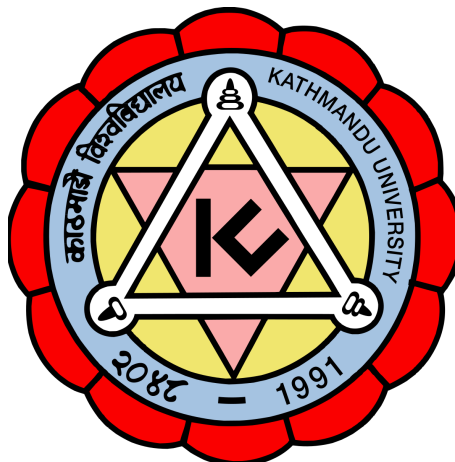


Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre



Lab Report 3
[Code No: COMP 342]

Submitted by:

Nischal Subedi (53)

Group: CS60 (III/I)

Submitted to:

Mr. Dhiraj Shrestha

Department of Computer Science and Engineering

Submission Date: 25/01/2026

Q1. Implement Cohen-Sutherland Line Clipping algorithm

The Cohen–Sutherland algorithm is a line clipping technique that uses **region codes** to determine the position of a line's endpoints relative to a rectangular clipping window. The 2D plane is divided into **nine regions**, and each endpoint is assigned a 4-bit code. Using these codes, the algorithm performs **trivial acceptance**, **trivial rejection**, or calculates intersection points to clip the line. It is simple and efficient for quickly determining whether a line lies completely inside or outside the clipping region.

```
from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *

# Clipping window
X_MIN, Y_MIN = -0.5, -0.5
X_MAX, Y_MAX = 0.5, 0.5

INSIDE = 0
LEFT = 1
RIGHT = 2
BOTTOM = 4
TOP = 8

def compute_code(x, y):
    code = INSIDE
    if x < X_MIN:
        code |= LEFT
    elif x > X_MAX:
        code |= RIGHT
    if y < Y_MIN:
        code |= BOTTOM
    elif y > Y_MAX:
        code |= TOP
    return code

def cohen_sutherland(x1, y1, x2, y2):
    code1 = compute_code(x1, y1)
    code2 = compute_code(x2, y2)
```

```

while True:
    if code1 == 0 and code2 == 0:
        return x1, y1, x2, y2
    if code1 & code2 != 0:
        return None

    code_out = code1 if code1 else code2

    if code_out & TOP:
        x = x1 + (x2 - x1) * (Y_MAX - y1) / (y2 - y1)
        y = Y_MAX
    elif code_out & BOTTOM:
        x = x1 + (x2 - x1) * (Y_MIN - y1) / (y2 - y1)
        y = Y_MIN
    elif code_out & RIGHT:
        y = y1 + (y2 - y1) * (X_MAX - x1) / (x2 - x1)
        x = X_MAX
    elif code_out & LEFT:
        y = y1 + (y2 - y1) * (X_MIN - x1) / (x2 - x1)
        x = X_MIN

    if code_out == code1:
        x1, y1 = x, y
        code1 = compute_code(x1, y1)
    else:
        x2, y2 = x, y
        code2 = compute_code(x2, y2)

def draw_window():
    glColor3f(1, 1, 1)
    glBegin(GL_LINE_LOOP)
    glVertex2f(X_MIN, Y_MIN)
    glVertex2f(X_MAX, Y_MIN)
    glVertex2f(X_MAX, Y_MAX)
    glVertex2f(X_MIN, Y_MAX)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)

```

```

draw_window()

# Original line
glColor3f(1, 0, 0)
glBegin(GL_LINES)
glVertex2f(-0.8, -0.2)
glVertex2f(0.8, 0.7)
glEnd()

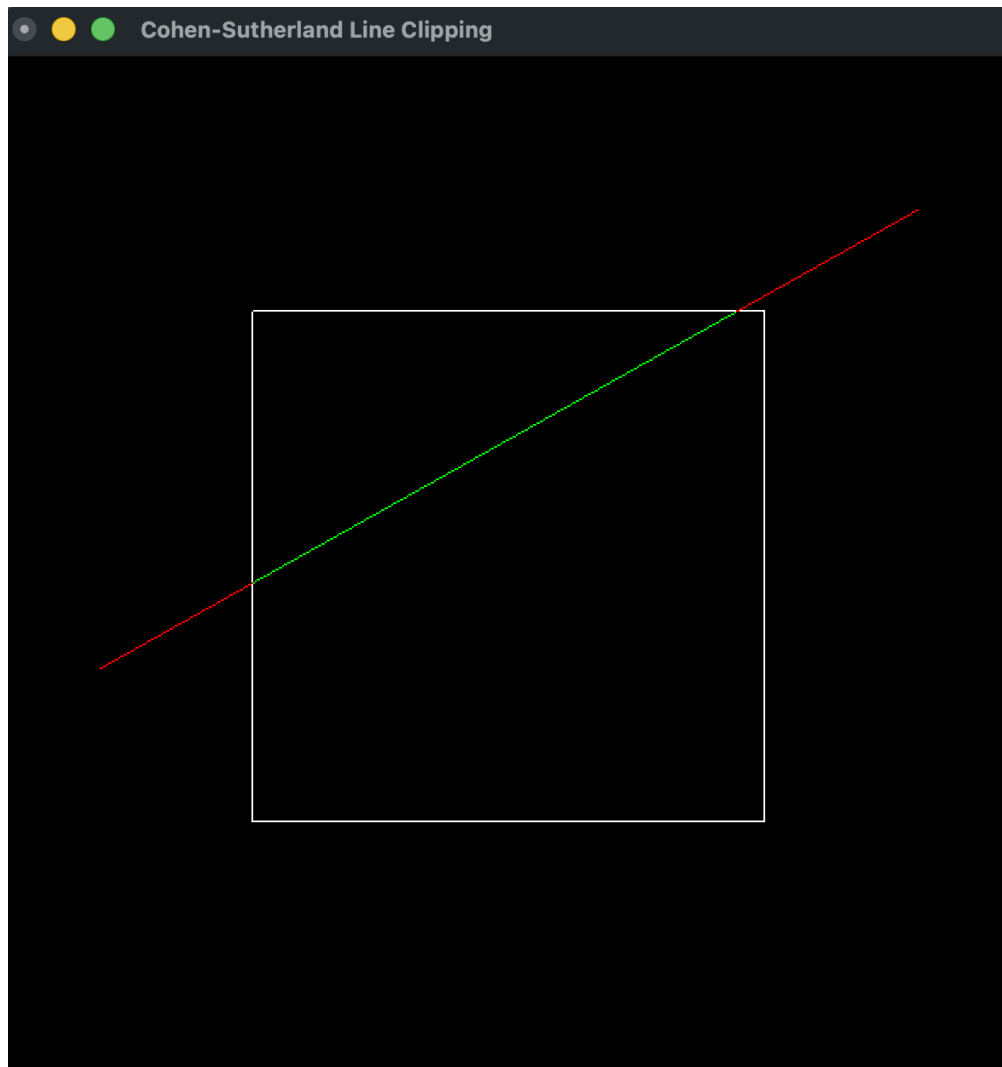
clipped = cohen_sutherland(-0.8, -0.2, 0.8, 0.7)
if clipped:
    glColor3f(0, 1, 0)
    glBegin(GL_LINES)
    glVertex2f(clipped[0], clipped[1])
    glVertex2f(clipped[2], clipped[3])
    glEnd()

glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(600, 600)
    glutCreateWindow(b"Cohen-Sutherland Line Clipping")
    gluOrtho2D(-1, 1, -1, 1)
    glutDisplayFunc(display)
    glutMainLoop()

if __name__ == "__main__":
    main()

```



Q2. Implement Liang-Barsky Line Clipping algorithm

The Liang–Barsky algorithm is a **parametric line clipping method** that represents a line using a parameterized equation. It uses a set of inequalities to compute the entering and leaving points of the line with respect to the clipping window. By calculating these values in a single pass, the algorithm efficiently determines the visible portion of the line. Compared to Cohen–Sutherland, it requires fewer computations and is more efficient for line clipping.

```
from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *
```

```

X_MIN, Y_MIN = -0.5, -0.5
X_MAX, Y_MAX = 0.5, 0.5

def liang_barsky(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1

    p = [-dx, dx, -dy, dy]
    q = [x1 - X_MIN, X_MAX - x1, y1 - Y_MIN, Y_MAX - y1]

    u1, u2 = 0.0, 1.0

    for pi, qi in zip(p, q):
        if pi == 0:
            if qi < 0:
                return None
            else:
                t = qi / pi
                if pi < 0:
                    u1 = max(u1, t)
                else:
                    u2 = min(u2, t)

    if u1 > u2:
        return None

    return (
        x1 + u1 * dx,
        y1 + u1 * dy,
        x1 + u2 * dx,
        y1 + u2 * dy
    )

def draw_window():
    glColor3f(1, 1, 1)
    glBegin(GL_LINE_LOOP)
    glVertex2f(X_MIN, Y_MIN)
    glVertex2f(X_MAX, Y_MIN)
    glVertex2f(X_MAX, Y_MAX)

```

```

    glVertex2f(X_MIN, Y_MAX)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    draw_window()

    glColor3f(1, 0, 0)
    glBegin(GL_LINES)
    glVertex2f(-0.8, 0.6)
    glVertex2f(0.9, -0.4)
    glEnd()

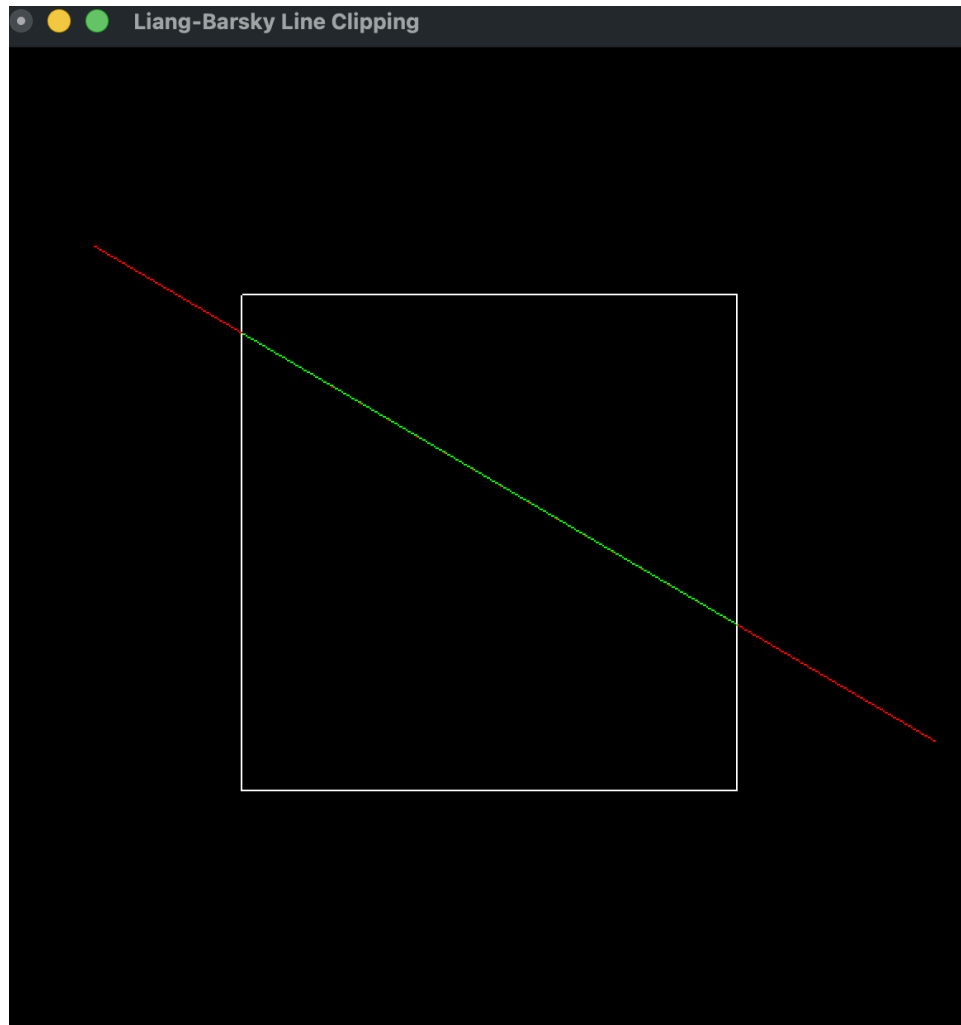
    clipped = liang_barsky(-0.8, 0.6, 0.9, -0.4)
    if clipped:
        glColor3f(0, 1, 0)
        glBegin(GL_LINES)
        glVertex2f(clipped[0], clipped[1])
        glVertex2f(clipped[2], clipped[3])
        glEnd()

    glFlush()

def main():
    glutInit()
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)
    glutInitWindowSize(600, 600)
    glutCreateWindow(b"Liang-Barsky Line Clipping")
    gluOrtho2D(-1, 1, -1, 1)
    glutDisplayFunc(display)
    glutMainLoop()

if __name__ == "__main__":
    main()

```



Q3. Implement Sutherland-Hodgman Polygon Clipping Algorithm

The Sutherland–Hodgman algorithm is used to clip a polygon against a **convex clipping window**. The polygon is processed edge by edge against each boundary of the clipping window. Vertices that lie inside the boundary are retained, and new intersection points are generated when polygon edges cross the clipping boundary. This process is repeated for all window edges, resulting in a clipped polygon that lies entirely within the viewing region.

```

from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *

X_MIN, Y_MIN = -0.5, -0.5
X_MAX, Y_MAX = 0.5, 0.5

def inside(p, edge):
    x, y = p
    if edge == "LEFT":
        return x >= X_MIN
    if edge == "RIGHT":
        return x <= X_MAX
    if edge == "BOTTOM":
        return y >= Y_MIN
    if edge == "TOP":
        return y <= Y_MAX

def intersect(p1, p2, edge):
    x1, y1 = p1
    x2, y2 = p2

    if edge == "LEFT":
        x = X_MIN
        y = y1 + (y2 - y1) * (X_MIN - x1) / (x2 - x1)
    elif edge == "RIGHT":
        x = X_MAX
        y = y1 + (y2 - y1) * (X_MAX - x1) / (x2 - x1)
    elif edge == "BOTTOM":
        y = Y_MIN
        x = x1 + (x2 - x1) * (Y_MIN - y1) / (y2 - y1)
    elif edge == "TOP":
        y = Y_MAX
        x = x1 + (x2 - x1) * (Y_MAX - y1) / (y2 - y1)

    return (x, y)

def sutherland_hodgman(polygon):
    edges = ["LEFT", "RIGHT", "BOTTOM", "TOP"]

```

```

for edge in edges:
    new_poly = []
    for i in range(len(polygon)):
        curr = polygon[i]
        prev = polygon[i - 1]

        if inside(curr, edge):
            if not inside(prev, edge):
                new_poly.append(intersect(prev, curr, edge))
            new_poly.append(curr)
        elif inside(prev, edge):
            new_poly.append(intersect(prev, curr, edge))
    polygon = new_poly

return polygon

def draw_window():
    glColor3f(1, 1, 1)
    glBegin(GL_LINE_LOOP)
    glVertex2f(X_MIN, Y_MIN)
    glVertex2f(X_MAX, Y_MIN)
    glVertex2f(X_MAX, Y_MAX)
    glVertex2f(X_MIN, Y_MAX)
    glEnd()

def display():
    glClear(GL_COLOR_BUFFER_BIT)
    draw_window()

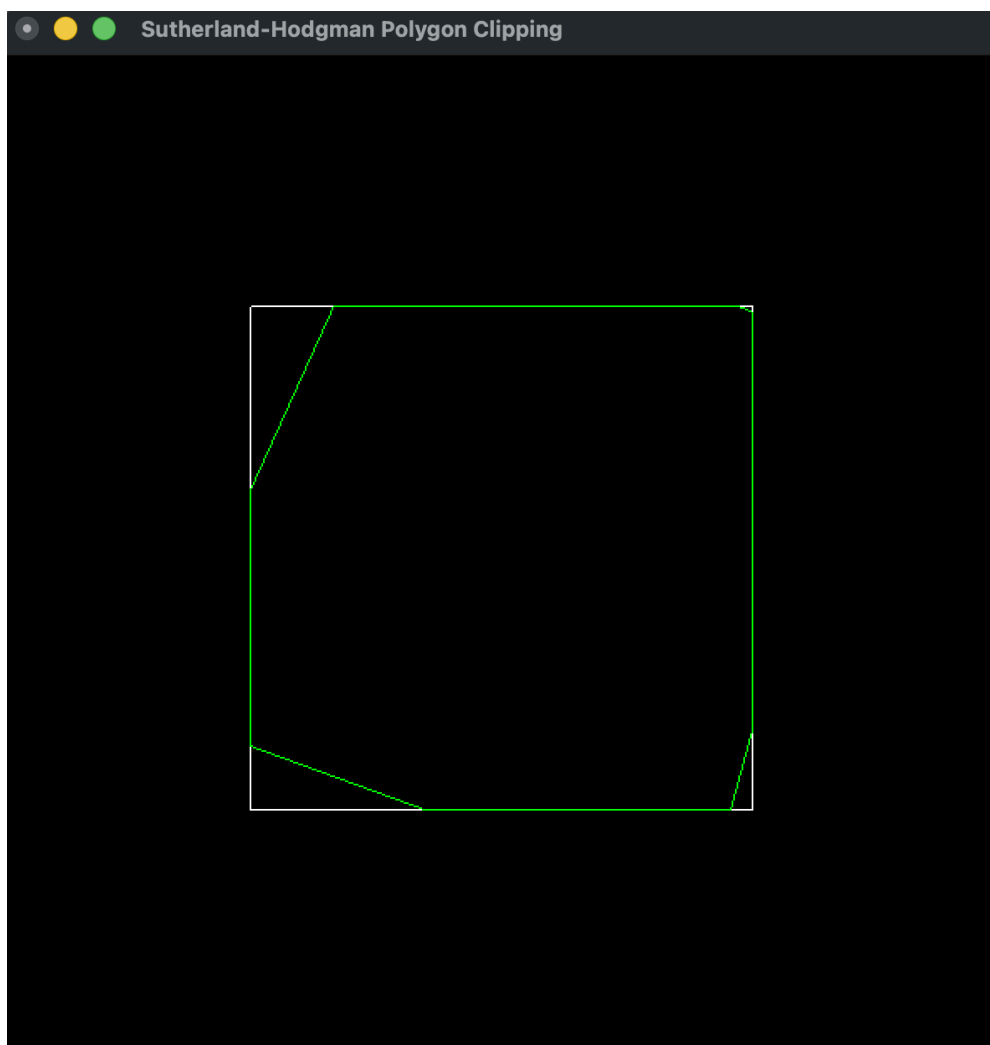
    polygon = [(-0.7, -0.3), (-0.2, 0.8), (0.7, 0.4), (0.4, -0.7)]
    clipped = sutherland_hodgman(polygon)

    glColor3f(0, 1, 0)
    glBegin(GL_LINE_LOOP)
    for x, y in clipped:
        glVertex2f(x, y)
    glEnd()

    glFlush()

```

```
def main():  
    glutInit()  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB)  
    glutInitWindowSize(600, 600)  
    glutCreateWindow(b"Sutherland-Hodgman Polygon Clipping")  
    gluOrtho2D(-1, 1, -1, 1)  
    glutDisplayFunc(display)  
    glutMainLoop()  
  
if __name__ == "__main__":  
    main()
```



Q4. Write a program to implement 2D reflection

Reflection creates a mirror image of an object across a specific axis. This implementation uses a matrix that negates the x-coordinate (to reflect across the Y-axis) while keeping the y-coordinate constant. This effectively "flips" the geometry to the opposite side of the Cartesian plane.

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def reflect_point(p):
    # Reflection over Y-axis: x becomes -x
    matrix = [[-1, 0, 0], [0, 1, 0], [0, 0, 1]]
    x, y = p
    return (matrix[0][0]*x, matrix[1][1]*y)

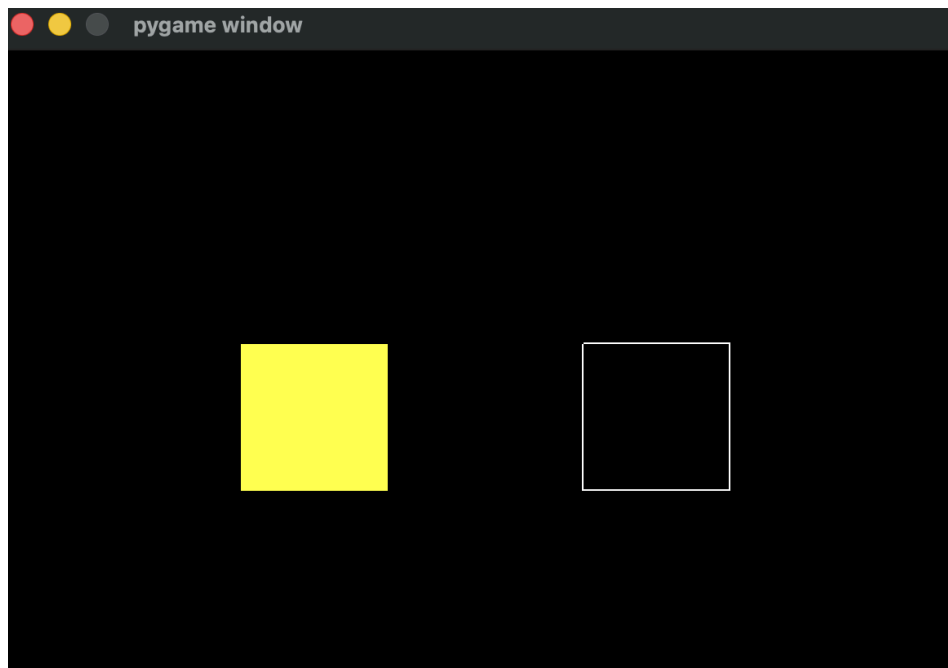
def main():
```

```

pygame.init()
pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
gluOrtho2D(-10, 10, -10, 10)
shape = [(2,1), (5,1), (5,4), (2,4)]
transformed = [reflect_point(p) for p in shape]
while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT: pygame.quit(); return
    glClear(GL_COLOR_BUFFER_BIT)
    glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
shape]; glEnd()
    glColor3f(1, 1, 0); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
    pygame.display.flip()

if __name__ == "__main__": main()

```



Q5. Write a program to implement 2D shearing

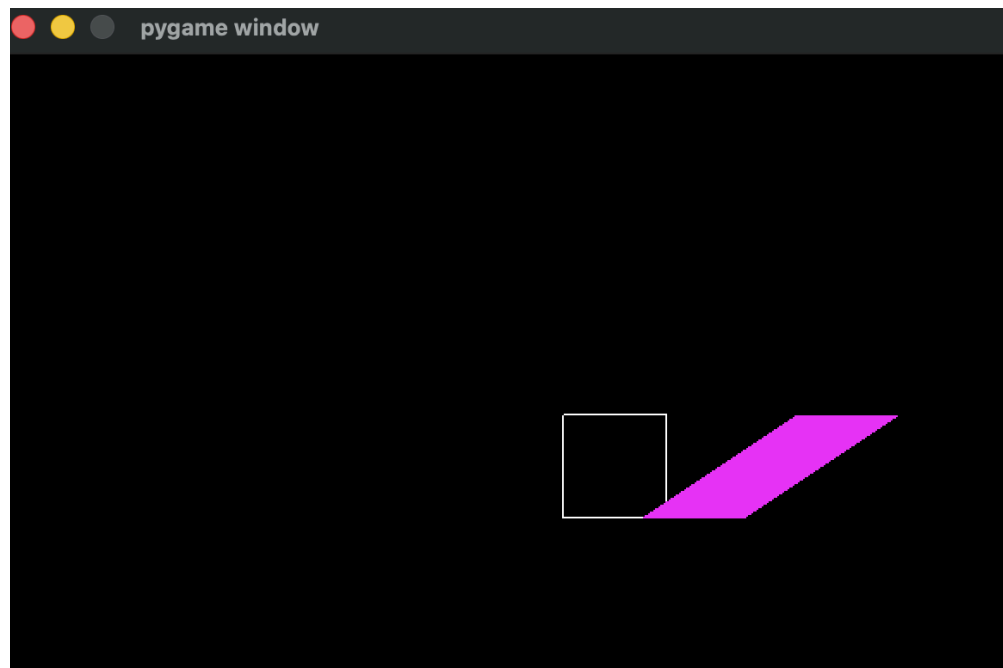
Shearing (or skewing) distorts the shape of an object such that it appears to be tilted. The program uses a shear matrix where the off-diagonal elements shx or shy are non-zero. This causes the coordinates of one axis to be modified based on the current value of the other axis, turning a rectangle into a parallelogram.

```
import pygame
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def shear_point(p, shx, shy):
    matrix = [[1, shx, 0], [shy, 1, 0], [0, 0, 1]]
    x, y = p
    new_x = x + shx * y
    new_y = shy * x + y
    return (new_x, new_y)
```

```
def main():
    pygame.init()
    pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
    gluOrtho2D(-10, 10, -10, 10)
    square = [(1,1), (3,1), (3,3), (1,3)]
    transformed = [shear_point(p, 1.5, 0) for p in square]
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT: pygame.quit(); return
        glClear(GL_COLOR_BUFFER_BIT)
        glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
square]; glEnd()
        glColor3f(1, 0, 1); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
        pygame.display.flip()

if __name__ == "__main__": main()
```



Q6. Write a program to implement composite transformations

This program demonstrates the power of the homogeneous coordinate system by combining four distinct transformations—**Scaling, Rotation, Shearing, and Translation**—into a single operation. By manually performing matrix multiplication ($T \cdot Sh \cdot R \cdot S$), we derive a single "Global Transformation Matrix." This allows all four steps to be applied to the vertices in a single mathematical pass, ensuring high efficiency.

```
import pygame, math
from pygame.locals import *
from OpenGL.GL import *
from OpenGL.GLU import *

def multiply_3x3(A, B):
    C = [[0,0,0], [0,0,0], [0,0,0]]
    for i in range(3):
        for j in range(3):
            for k in range(3):
```

```

        C[i][j] += A[i][k] * B[k][j]

    return C

def main():
    pygame.init()
    pygame.display.set_mode((600, 600), DOUBLEBUF | OPENGL)
    gluOrtho2D(-10, 10, -10, 10)

    # 1. Scaling Matrix
    S = [[0.5, 0, 0], [0, 0.5, 0], [0, 0, 1]]

    # 2. Rotation Matrix (30 deg)
    a = math.radians(30); c, s = math.cos(a), math.sin(a)
    R = [[c, -s, 0], [s, c, 0], [0, 0, 1]]

    # 3. Shear Matrix
    Sh = [[1, 0.5, 0], [0, 1, 0], [0, 0, 1]]

    # 4. Translation Matrix
    T = [[1, 0, 2], [0, 1, -3], [0, 0, 1]]

    # Composite = T * Sh * R * S (Applied right to left)
    comp = multiply_3x3(T, multiply_3x3(Sh, multiply_3x3(R, S)))

    shape = [(0,0), (2,0), (2,2), (0,2)]
    transformed = []
    for x, y in shape:
        nx = comp[0][0]*x + comp[0][1]*y + comp[0][2]*1
        ny = comp[1][0]*x + comp[1][1]*y + comp[1][2]*1
        transformed.append((nx, ny))

    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT: pygame.quit(); return
        glClearColor(GL_COLOR_BUFFER_BIT)
        glColor3f(1, 1, 1); glBegin(GL_LINE_LOOP); [glVertex2f(p[0], p[1]) for p in
shape]; glEnd()
        glColor3f(0, 1, 1); glBegin(GL_POLYGON); [glVertex2f(p[0], p[1]) for p in
transformed]; glEnd()
        pygame.display.flip()

if __name__ == "__main__": main()

```



